Feedback Loop — reviewer signals tune next swarm run per category

SPEC-SW-010 v0.1 DRAFT Jeremie Nunez

1 Context

The sweep subsystem runs a fan-out of cheap nano workers over the domain catalogue, each producing *suggestions* that a human reviewer either accepts or rejects. Without a feedback channel, the swarm repeats the same low-signal suggestions cycle after cycle: the reviewer ends up rejecting the same kind of false positive every week. This spec defines the closed loop that turns reviewer decisions into per-category prompt conditioning for the next swarm run, so rejection rates decay over time without any manual prompt editing.

2 Goals

- Persist every accept/reject decision with its category, target entity, and optional reviewer note.
- Inject the most recent decisions back into the nano prompt for the *same* category on the next run, within one sweep cycle.
- Make the effect observable: per-category rejection rate must be queryable as a time series and must trend down when the signal is informative.
- Bound the feedback block size so prompt tokens stay within the configured budget regardless of feedback volume.
- Fail open: if the feedback store is unreachable, the swarm runs without conditioning rather than blocking.

3 Non-Goals

Non-Goal

This spec does not cover: (a) how suggestions are generated (SPEC-SW-001 Nano Sweep), (b) the reviewer UI surface, (c) cross-category feedback (a reject in `price_anomaly` does not condition `missing_data`), (d) model fine-tuning or embedding-based recall of past feedback — only literal injection of recent decisions is in scope, (e) long-term feedback archiving beyond the rolling window.

4 Invariants

Invariant

A reject decision recorded before sweep cycle N starts *must* be reflected in the prompt used for its category during cycle N. There is no grace cycle: one cycle of latency is the upper bound, not a target.

Invariant

Feedback is category-scoped: the prompt for category C in cycle N only embeds decisions whose stored category equals C. No cross-leakage between categories.

Invariant

The injected feedback block is bounded: at most `MAX_FEEDBACK_LINES` entries per category per batch, ordered most-recent-first, truncated by reviewer note length to fit `MAX_FEEDBACK_CHARS`.

Invariant

Feedback store unavailability degrades gracefully: the sweep still runs, with an empty feedback block, and emits a warning log — it never throws into the caller.

Invariant

Per-category rejection rate is a monotonically defined observable: for any window $[t_0, t_1]$ and category C, $\texttt{rejectionRate}(C, t_0, t_1) = \texttt{rejected}(C)/(\texttt{rejected}(C) + \texttt{accepted}(C))$, computed from persisted decisions only.

5 Interface

Public surface exposed by the feedback subsystem. Consumers (nano sweep service, admin dashboards) depend on these signatures; internal storage layout is free to change.

```
// Persisted decision record
export interface ReviewerDecision {
  id: string;
  category: SweepCategory;
  entityName: string | null;      // e.g. appellation / dataset / segment
  wasAccepted: boolean;
  reviewerNote: string | null;
  decidedAt: Date;
}

// Loaded into the prompt builder
export interface PastFeedback {
  category: SweepCategory;
  entityName: string | null;
  wasAccepted: boolean;
  reviewerNote: string | null;
}

export interface FeedbackRepository {
  recordDecision(input: Omit<ReviewerDecision, "id" | "decidedAt">):
     Promise<ReviewerDecision>;
  loadPastFeedback(opts?: {
    category?: SweepCategory;
    limit?: number;           // defaults to MAX_FEEDBACK_LINES
  }): Promise<PastFeedback[]>;
  rejectionRate(
    category: SweepCategory,
    window: { from: Date; to: Date },
  ): Promise<{ rejected: number; accepted: number; rate: number }>;
}

// Prompt builder contract - pure function, easy to test.
export function buildFeedbackBlock(
  category: SweepCategory,
  feedback: PastFeedback[],
): string;
```

6 Scenarios

Scenario: Single reject is reflected in the next cycle

Given the reviewer rejects a suggestion in category `price_anomaly` for entity `E` with note `"price is within the historical band"` at t_0
And no sweep is currently in flight
When a new sweep cycle starts at $t_1 > t_0$
Then the nano prompt generated for any batch containing entity `E` in category `price_anomaly` contains a feedback line referencing `E`, marked `REJECTED`, with the reviewer note verbatim

Scenario: Category isolation

Given a reject recorded for category `missing_data` on entity `E`
When the next sweep runs and produces a batch targeting entity `E`
Then the prompt for category `price_anomaly` on that batch contains no feedback line for `E`
And the prompt for category `missing_data` on that batch does contain the feedback line

Scenario: Feedback block is bounded

Given 500 rejects exist for category `enrichment`
When the next sweep cycle builds a prompt for that category
Then the injected feedback block contains at most `MAX_FEEDBACK_LINES` lines
And the entries are ordered most-recent-first
And the serialized block size does not exceed `MAX_FEEDBACK_CHARS` characters

Scenario: Feedback store unavailable

Given the feedback repository raises on `loadPastFeedback`
When a sweep cycle starts
Then the sweep completes with an empty feedback block
And a warning is logged with the underlying error
And no exception escapes the service boundary

Scenario: Rejection rate decay is observable

Given 4 consecutive weekly sweep cycles, each preceded by at least one reject in category `price_anomaly` with informative reviewer notes
When the rejection rate is computed for each weekly window
Then the rate for week $N+1$ is less than or equal to the rate for week N, within a documented tolerance, for categories where reviewer notes were informative
And the raw `rejected` and `accepted` counts are exposed per window

7 Acceptance Criteria

Acceptance Criterion AC-1

`recordDecision` persists all fields; a subsequent `loadPastFeedback` with a matching category returns the record.

Acceptance Criterion AC-2

For a reject recorded at t_0, the first sweep cycle started after t_0 produces, for the matching category and entity, a prompt whose feedback block contains a line tagged `REJECTED` with the reviewer note.

Acceptance Criterion AC-3

Given 1000 stored decisions across all categories, `loadPastFeedback({category: C})` returns at most `MAX_FEEDBACK_LINES` entries, all with category equal to C.

Acceptance Criterion AC-4

`buildFeedbackBlock(C, feedback)` emits no line belonging to a category other than C, given mixed-category input.

Acceptance Criterion AC-5

When the feedback repository throws, the sweep service completes successfully with a non-empty result and an empty feedback block; a warning log line is emitted.

Acceptance Criterion AC-6

`rejectionRate(C, window)` returns values consistent with `rate` = `rejected`/(`rejected` + `accepted`) and 0 when the window contains no decisions.

Acceptance Criterion AC-7

In a fixture simulating 4 cycles with informative rejects for category C, the computed rejection rate for cycle 4 is strictly less than that of cycle 1.

8 Traceability

AC	Test file	Test name
AC-1	`packages/sweep/tests/unit/feedback-repository.spec.ts`	`records and reloads a decisio`
AC-2	`packages/sweep/tests/integration/feedback-loop.spec.ts`	`reject surfaces in next cycle`
AC-3	`packages/sweep/tests/unit/feedback-repository.spec.ts`	`loadPastFeedback caps at MAX_`
AC-4	`packages/sweep/tests/unit/feedback-prompt.spec.ts`	`buildFeedbackBlock is categor`
AC-5	`packages/sweep/tests/integration/feedback-loop.spec.ts`	`repository failure degrades g`
AC-6	`packages/sweep/tests/unit/feedback-repository.spec.ts`	`rejectionRate math`
AC-7	`packages/sweep/tests/integration/feedback-decay.spec.ts`	`rejection rate decays across`

9 Open Questions

- Should accepted decisions also be injected, or only rejects? Current draft injects both, tagged accordingly.
- What is the right default for `MAX_FEEDBACK_LINES` (current proposal: 5)?
- Do we need an explicit `expiresAt` on decisions, or is the rolling most-recent window sufficient?
- Should reviewer notes be truncated per-entry, or the block truncated as a whole once the char budget is hit?